difference between the two, so you'll need a different justification if you want to prefer that style quirk.

Ruby will concatenate string literals that are next to each other if there is no operator between them:

```
"This" "is" "just" "one" "string" # => "Thisisjustonestring"
```

Ruby provides an alternative to single and double-quote delimiters, which comes in handy sometimes when the string you want to quote contains the delimiter you need. The syntax is %q or %Q followed by a delimiter character. The %q delimiter is equivalent to single-quote, and %Q is equivalent to double quote. In fact, the Q is optional, just a % followed by the delimiter is equivalent to double-quote:

```
%q/general single-quoted string/   # => general single-quoted string
%Q!general double-quoted string!   # => general double-quoted string
%Q{Seconds/day: #{24*60*60}}       # => Seconds/day: 86400
%!general double-quoted string!    # => general double-quoted string
%{Seconds/day: #{24*60*60}}        # => Seconds/day: 86400
```

Notice that the delimiter changes from line to line. Whatever character follows the initial *q* or *Q* is the delimiter—this also goes for other variants on the flexible delimiter syntax we will see. If the character is an opening bracket [, brace {, parenthesis (, or less-than sign <, the string is read until the matching close symbol is found. Otherwise, the string is read until the next occurrence of the same delimiter. The delimiter can be any nonalphanumeric or nonmultibyte character. Current code style guidelines will suggest that you stick to parenthesis as the string delimiter, %q().

Finally, you can construct a string using a *here document*, or *heredoc*. A heredoc allows you to build a multi-line string.

```
string = <<END_OF_STRING
  The body of the string is the input lines up to
  one starting with the same text that followed the '<<'
END_OF_STRING
```

A here document consists of lines in the source up to but not including the terminating string that you specify after the << characters. Normally, this terminator must start in column one. However, if you put a minus sign after the << characters, you can indent the terminator:

```
string = <<-END_OF_STRING
The body of the string is the input lines up to
one starting with the same text that followed the '<<'
  END_OF_STRING
```

And if you put a tilde after the << characters you can indent the text. Well, you can always indent the text, but if you use a ~, then Ruby will remove the indentation spaces from the beginning of each line, making it easier to lay out a long string:

```
def a_long_string
  <<~END_OF_STRING
    Faster than a speeding bullet, more powerful than
    a locomotive, able to leap tall buildings in a single
    bound—look, up there in the sky, it's a bird, it's a
    plane, it's Superman!
  END_OF_STRING
```